

Comprehensive Technical Documentation:

VisionCheckAI Backend Architecture

1. Introduction and Project Objective

The fundamental objective of the VisionCheckAI backend is to serve as the central orchestration engine for an automated quality control platform within manufacturing environments. While the computer vision models classify physical parts into overarching categories (such as Defective and Non-Defective), the backend provides the critical infrastructure required to handle image ingestion, securely manage user roles, and persistently store inspection records. This transition from isolated AI scripts to a robust web API accelerates the manufacturing pipeline by enabling real-time dashboarding, remote supervisory review, and consistent, objective quality tracking.

2. Architecture and Technology Selection Rationale

2.1 Why ASP.NET Core?

When selecting a backend framework for an industrial application, there is an inherent trade-off between rapid prototyping and enterprise-level reliability. While lightweight frameworks (like Flask or FastAPI) are often used alongside Python-based ML models, they can struggle with complex dependency injection, out-of-the-box security, and rigid object-relational mapping required for large-scale manufacturing systems.

ASP.NET Core, conversely, was designed for high-performance, concurrent enterprise applications. It provides a highly advanced, state-of-the-art request pipeline that can efficiently handle heavy multipart file uploads (such as high-resolution inspection images) without blocking the main thread. By separating the web server (C#) from the inference engine (Python), the architecture avoids unnecessary computational overhead in the API layer and prevents memory leaks inherent in running large ML models inside a web process.

2.2 Database and Storage Decisions

We employed SQLite via Entity Framework (EF) Core for the persistence layer. SQLite was chosen to balance operational footprint with stable data retention. It ensures zero-configuration, serverless database deployment, which is ideal for a localized factory floor deployment where setting up a dedicated SQL Server instance might introduce

unnecessary IT overhead.

3. Data Models and Entity Relationships

The dataset (database) is structured into four distinct entities representing the core operational domains of the platform.

Entity	Primary Role	Relationships	Broad Category
Users	Manages authentication and roles (Inspector, Supervisor).	Independent	Security
Products	Stores catalog of manufactured items (e.g., M8 Hex Nut).	Referenced by Inspections	Inventory
Inspections	Records individual image uploads, AI results, and timestamps.	One-to-Many with Defects	Core Operation
Defects	Logs specific bounding boxes and confidence scores for anomalies.	Belongs to Inspection	Analytics

As evident from the schema, the dataset exhibits a highly relational structure necessary for tracking the full lifecycle of a manufactured part, from the moment it is photographed by an operator to the moment a supervisor reviews the AI's prediction.

4. Comprehensive Request Pipeline

The backend strategy was separated into specific stages to handle incoming HTTP requests efficiently. This phased approach is a best practice in backend engineering.

4.1 Stage 1: Security and Authentication

In the first stage, requests are filtered through a JWT (JSON Web Token) middleware. The parameters of the incoming request's `Authorization` header are validated against a symmetric security key. Only authenticated users with specific claims (such as `Role: Supervisor`) are granted access to sensitive endpoints like overriding AI predictions.

4.2 Stage 2: Inference Orchestration

After the security layer achieves a baseline level of competency in validating the user, the core business logic is executed. For image uploads, the backend subtly adjusts its execution flow to spawn an external Python process (`predict.py`). It passes the image path to the ML model, awaits the JSON output, and gracefully falls back to simulated predictions if the AI environment is unavailable, ensuring the production line never halts due to software latency.

5. Detailed Analytics Metrics

Evaluating an industrial production line requires more than just standard image storage. The dashboard controllers provide deep insights into the manufacturing process.

5.1 Calculation Metrics Explained

- **Total Inspections:** The absolute number of samples processed by the system.
- **Defect Rate:** The ratio of defective predictions to the total inspections. High defect rates trigger immediate supervisory alerts.
- **Defects by Category:** A grouped count of specific anomalies (e.g., Rusting vs. Scratches), which is the most critical metric for identifying specific machinery failures on the factory floor.
- **Daily Trend:** A rolling 7-day progression of inspection outcomes (Total, Pass, and Defective) to monitor quality over time.

6. Real-World API Predictions and Inference

6.1 Analyzing Workflows

By visually inspecting the system's operational flow, we divide the results into two categories: * **Automated Pass:** In these cases, the model accurately assigns the 'Excellent' class with remarkably high confidence, and the backend automatically archives the image without requiring human intervention. * **Human-in-the-Loop Review:** Analyzing defective parts is crucial for debugging the manufacturing line. When a fracture is detected, the backend holds the record in a 'Pending' state until a human supervisor confirms the result, acting as the literal bridge between theoretical data science and practical quality assurance.

7. References

1. ASP.NET Core Documentation: <https://learn.microsoft.com/en-us/aspnet/core>
2. Entity Framework Core SQLite Provider:
<https://learn.microsoft.com/en-us/ef/core/providers/sqlite>